

Guía de estudio: VentasFx

Cómo explicar la programación funcional de tu proyecto, lección por lección

Alumno	Alan Rodríguez · No. de control 23940396 · 7 AS
Materia	Programación Lógica y Funcional · Unidad 2
App en línea	ventasfx-alan.streamlit.app
Código	github.com/Twenty-source/ventasfx (tecla <code>.</code> para abrir el editor web)

Cómo usar esta guía

1. Lee una lección a la vez. Cada una tiene: la idea en una frase, el código real, por qué importa y lo que conviene decir.
2. Resuelve el ejercicio **sin mirar las soluciones** y contéstalo en voz alta, como si hablaras con el docente.
3. Compara con las **soluciones** al final y repasa lo que falló.
4. Abre el archivo en github.dev y busca la marca `[PF ...]` de la lección para verla en el código.
5. Antes de la revisión, lee la **hoja de repaso rápido** de la última página.

Contenido

#	Lección	Archivo principal	Temas
1	La idea general y los datos inmutables	<code>core/modelos.py</code>	Inmutabilidad, 2.1
2	Funciones puras	<code>core/funciones.py</code>	Pureza, efectos
3	Funciones como valores, orden superior y predicados	<code>core/predicados.py</code>	2.2, 2.4, lambda
4	map, filter y reduce	<code>core/reportes.py</code> , <code>core/pipeline.py</code>	2.5, comprensiones
5	Recursividad y árboles	<code>core/arbol.py</code>	Recursividad, 2.6
6	Evaluación perezosa y generadores	<code>core/perezoso.py</code>	2.3, 2.7
7	El estado de la app y las preguntas difíciles	<code>ui/estado.py</code>	Arquitectura, defensa

Lección 1 · La idea general y los datos inmutables

El programa **no cambia datos: crea datos nuevos a partir de los anteriores.**

Piensa en un ticket impreso: si te equivocas no lo borras con corrector, imprimes uno nuevo y el viejo sigue existiendo. Así trabaja todo VentasFx.

Cómo se logra en el código

```
@dataclass(frozen=True)
class Producto:
    sku: str
    nombre: str
    precio: float
    ...

class Venta(NamedTuple):
    folio: str
    fecha: date
    cantidad: int
    ...
```

- `frozen=True` congela el producto: `producto.precio = 10` lanza un error.
- `NamedTuple` es una tupla con nombres; las tuplas ya son inmutables.

Si se necesita otro precio, se crea **otro** producto:

```
def ajustar_precio(producto, porcentaje):
    return replace(producto, precio=producto.precio * (1 + porcentaje))
```

Por qué importa

1. **Nadie daña los datos por accidente**, aunque todas las pantallas compartan las mismas ventas.
2. **Deshacer es gratis**: como nada se borra, volver atrás es regresar al valor anterior.
3. **Es más fácil razonar**: si un dato no cambia, no hay que rastrear quién lo modificó.

Tipos de datos (tema 2.1)

Tipo	Ejemplo en el código	Por qué ese tipo
<code>tuple</code>	<code>VENEDORES</code> , <code>CATALOGO</code>	Colección que no debe cambiar
<code>dict</code>	<code>INDICE_PRODUCTOS</code>	Buscar un producto por su SKU al instante
<code>set</code>	<code>CATEGORIAS_RAIZ</code>	Categorías sin repetir
<code>float</code> · <code>int</code> · <code>bool</code> · <code>str</code>	<code>precio</code> · <code>stock</code> · <code>activo</code> · <code>nombre</code>	Datos simples de cada producto

RESPUESTA MODELO · "¿POR QUÉ TUPLA Y NO LISTA PARA CATALOGO?"

"Usé tupla porque el catálogo son datos que no deben cambiar. Python impide modificarla, así que ninguna parte del programa puede alterar los datos por accidente, aunque todas las pantallas los compartan. Además deja clara la intención: es una colección fija. Si necesito un catálogo distinto, por ejemplo con precios ajustados, creo uno nuevo con `map` y el original queda igual."

CUIDADO

La tupla ocupa un poco menos de memoria que una lista, pero **no es la razón principal**. Si dices "es más eficiente", el docente puede preguntar "¿en qué?". La razón fuerte es la seguridad y la intención.

Búscalo en el código: [\[PF inmutable\]](#) · [\[PF 2.1 tipos\]](#)

Lección 2 · Funciones puras

Una función pura es como una calculadora: **misma entrada, misma salida, y no toca nada fuera de ella.**

El contraejemplo (diapositiva 6, `core/didactico.py`)

```
total = 0

def agregar(x):          # IMPURA
    global total
    total += x
    return total

agregar(5) # 5
agregar(5) # 10 ← misma entrada, distinta salida
```

Es impura por dos razones: **depende** de algo externo (`total`) y **modifica** algo externo.

Las funciones puras del sistema (`core/funciones.py`)

```
def importe_bruto(venta):
    return venta.precio * venta.cantidad

def subtotal(venta):
    return importe_bruto(venta) - monto_descuento(venta)

def total_venta(venta):
    return redondear(subtotal(venta) + calcular_iva(subtotal(venta)))
```

- **Solo usan lo que reciben:** no leen variables que cambien, no imprimen, no escriben archivos.
- **Se construyen unas con otras:** `total_venta` usa `subtotal`, que usa `importe_bruto`. Piezas pequeñas que se componen.

Las dos condiciones (las dos deben cumplirse)

Condición	<code>agregar(x)</code>	<code>registrar_y_calcular(venta, historial)</code>
Misma entrada → misma salida	❌ (5, luego 10)	✅ (siempre el mismo total)
No modifica nada de afuera	❌ (cambia <code>total</code>)	❌ (hace <code>historial.append</code>)

Basta con que falle una para que sea impura. Modificar algo externo se llama **efecto secundario**. La versión pura devuelve un valor nuevo:

```
def registrar_y_calcular(venta, historial):
    return historial + (venta,), total_venta(venta)
```

Es lo que hace la app con el inventario: `actualizar_stock` devuelve `{**inventario, sku: ...}`, un diccionario nuevo.

Por qué importa

1. **Se prueban fácil:** hay 47 pruebas; basta dar una entrada y comparar la salida.
2. **Se reutilizan:** `total_venta` se usa en Dashboard, Reportes, Ventas y Carga masiva.
3. **Se pueden memoizar:** guardar resultados solo es válido si la salida depende únicamente de la entrada (lección 5).

PREGUNTA TRAMPA

"`calcular_iva` usa `TASA_IVA`, que es global. ¿No es impura?" — No: `TASA_IVA` es una **constante**, nadie la modifica. El problema de `agregar` no es usar algo de afuera, sino usar algo de afuera **que cambia**.

RESPUESTA MODELO · "¿QUÉ ES UNA FUNCIÓN PURA?"

"Es una función que con la misma entrada siempre da la misma salida y no modifica nada fuera de ella. Todas las reglas de negocio de mi sistema son así: el IVA, el total, la utilidad, las comisiones. Por eso las puedo probar con `pytest` sin preparar nada y reutilizarlas en todas las pantallas."

Búscalo en el código: [\[PF pura\]](#) · [\[PF efecto\]](#)

Lección 3 · Funciones como valores, orden superior y predicados

Las funciones son **valores**: se guardan, se pasan y se devuelven, igual que un número.

Idea 1 · Primera clase: funciones dentro de un diccionario

```
OPERADORES = {  
    ">": operator.gt,  
    ">=": operator.ge,  
    "==" : operator.eq,  
    ...  
}
```

`operator.gt(10, 3)` es lo mismo que `10 > 3`: el símbolo convertido en función (tema 2.4). El usuario elige `">"` en pantalla y el programa obtiene la función.

Idea 2 · Predicado = función que responde Sí / No

```
def es_par(x):  
    return x % 2 == 0      # True o False
```

Idea 3 · Orden superior: funciones que fabrican funciones

```
def regla(campo, simbolo, valor):  
    extraer = CAMPOS[campo]      # p. ej. attrgetter("cantidad")  
    comparar = OPERADORES[simbolo] # p. ej. operator.ge  
    return lambda venta: comparar(extraer(venta), valor)
```

`regla()` **no filtra nada: devuelve una función nueva**. La `lambda` es una función sin nombre que "recuerda" `comparar`, `extraer` y `valor` aunque `regla()` ya terminó (eso se llama *closure*).

Idea 4 · Combinar predicados

```
def y_(*predicados):  
    return lambda x: all(p(x) for p in predicados)
```

Recibe varias funciones y devuelve una sola. La pantalla Ventas une todos los filtros en **un único predicado** y hace un solo `filter`.

La analogía del portero

Código	Analogía
<code>caros = regla("precio", ">", 10000)</code>	Contratas al portero y le das la instrucción
<code>caros</code>	Es el portero (el predicado)
<code>caros(laptop)</code>	El portero revisa a una persona → <code>True</code>
<code>filter(caros, ventas)</code>	La fila completa: pasa solo quien recibe "sí"

RESPUESTA MODELO · "¿DÓNDE USAS FUNCIONES DE ORDEN SUPERIOR?"

"En los filtros de Ventas. `regla()` recibe un campo, un operador y un valor, y devuelve un predicado nuevo. `y_()`, `o_()` y `no_()` reciben predicados y devuelven uno combinado. Al final tengo una sola función que le paso a `filter`. Los operadores vienen del módulo `operator`, guardados en un diccionario."

Búscalo en el código: [\[PF 2.2 primera clase\]](#) · [\[PF 2.2 orden superior\]](#) · [\[PF 2.2 lambda\]](#) · [\[PF 2.4 operator\]](#) · [\[PF 2.4 predicado\]](#)

Lección 4 · map, filter y reduce

map transforma · filter selecciona · reduce acumula (diapositiva 15)

Función	Qué hace	Entra → Sale
<code>map(f, datos)</code>	Aplica <code>f</code> a cada elemento	5 ventas → 5 totales
<code>filter(p, datos)</code>	Se queda con los que cumplen <code>p</code>	5 ventas → 2 ventas
<code>reduce(f, datos, inicial)</code>	Combina todos en uno	5 totales → 1 número

Ejemplo 1 · El más simple (`core/reportes.py`)

```
def total_general(ventas):  
    return reduce(operator.add, map(total_venta, ventas), 0.0)
```

De adentro hacia afuera: `map` convierte cada venta en su total; `reduce` los suma empezando en 0.0:

```
inicio:          0.0  
0.0    + 1250.5 = 1250.5  
1250.5 + 899   = 2149.5  
2149.5 + 24999 = 27148.5 ← resultado
```

Ejemplo 2 · Los 5 indicadores del Dashboard en una pasada

```
def _acumular_resumen(acc, venta):  
    n, total, unidades, ganancia, mayor = acc  
    t = total_venta(venta)  
    return (n + 1, total + t, unidades + venta.cantidad,  
            ganancia + utilidad(venta), max(mayor, t))  
  
reduce(_acumular_resumen, ventas, (0, 0.0, 0, 0.0, 0.0))
```

El acumulador es una **tupla de 5 valores**. En cada paso se recibe la tupla actual y una venta, y se devuelve **una tupla nueva**. Una sola vuelta sobre los datos produce todos los indicadores.

Ejemplo 3 · El pipeline de la pantalla Ventas

```
60,000 ventas → filter(predicado) → sorted → map(total_venta) → reduce(add) → $ total
```

Es la frase de la diapositiva 32: **DATOS → FILTER → MAP → REDUCE → RESULTADO**.

`ejecutar_pipeline` usa `itertools.accumulate` (un reduce que conserva cada paso) para dibujar las tarjetas.

Comprensiones: map + filter "a la Python"

```
list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, numeros))) # funcional
[x ** 2 for x in numeros if x % 2 == 0]                          # comprensión
```

`x ** 2` es el **map**; `if x % 2 == 0` es el **filter**. La diapositiva 20 advierte no abusar de `lambda` cuando la comprensión se lee mejor.

DETALLE HONESTO QUE PUEDE SALIR

`agrupar()` usa un `reduce` cuyo acumulador es un diccionario que **sí se modifica** (`_sumar_en`). Es una "mutación local controlada": el diccionario se crea dentro de la función y nadie más lo ve, así que `agrupar` sigue siendo pura hacia afuera. Copiarlo en cada paso sería muy lento sin ningún beneficio. Está marcado como [\[PF efecto\]](#).

EJERCICIO 4

Tienes ventas con totales **\$500, \$1,500, \$3,000 y \$800**. ¿Cómo obtienes la suma de los totales mayores a \$1,000? Di qué función hace cada paso y el resultado.

Búscalo en el código: [\[PF 2.5 map\]](#) · [\[PF 2.5 filter\]](#) · [\[PF 2.5 reduce\]](#) · [\[PF 2.5 comprensión\]](#)

Lección 5 · Recursividad y árboles

Una función recursiva **se llama a sí misma con un problema más pequeño** hasta llegar a un **caso base** que se responde directo.

Las dos partes obligatorias (diapositiva 22)

```
def factorial(n):
    if n == 0:                # CASO BASE: se responde sin llamarse otra vez
        return 1
    return n * factorial(n - 1) # CASO RECURSIVO: problema más pequeño
```

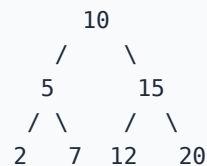
```
factorial(3)
3 * factorial(2)
3 * 2 * factorial(1)
3 * 2 * 1 * factorial(0)
3 * 2 * 1 * 1 = 6
```

"¿Dónde está el `for`?" No hace falta: la repetición la hacen las llamadas. **Sin caso base** la función se llamaría para siempre y Python lanzaría `RecursionError`.

Árboles: la estructura naturalmente recursiva (diapositivas 23 y 24)

Cada nodo es una tupla (`valor`, `izquierda`, `derecha`) y el árbol vacío es `None`. Cada lado es, a su vez, un árbol completo: por eso se procesa con recursividad.

```
ARBOL_EJEMPLO = (
    10,
    (5, (2, None, None), (7, None, None)),
    (15, (12, None, None), (20, None,
None))),
)
```



```
def suma_arbol(arbol):
    if arbol is None:        # caso base: árbol vacío suma 0
        return 0
    valor, izquierda, derecha = arbol
    return valor + suma_arbol(izquierda) + suma_arbol(derecha)

suma_arbol(ARBOL_EJEMPLO) # 71
```

Dónde se usa en VentasFx (core/arbol.py)

Función	Qué hace	Concepto clave
<code>insertar</code>	Agrega un valor y devuelve un árbol nuevo	Solo se recrean los nodos del camino; la rama no tocada se comparte (segura porque es inmutable)
<code>construir_bst</code>	Inserta todos los productos uno por uno	<code>reduce</code> con el árbol como acumulador
<code>construir_balanceado</code>	Raíz = elemento de en medio; cada mitad, un subárbol	Divide y vencerás: altura 6 contra 12 con los mismos 34 productos
<code>inorden</code>	Recorre izquierda → raíz → derecha	Generador recursivo (<code>yield from</code>); en un BST da los precios ordenados
<code>buscar_rango</code>	Productos entre dos precios	Poda: no visita ramas que no pueden tener resultados
<code>construir_arbol_categorias</code>	Catálogo → Electrónica → Cómputo → Laptops → producto	Árbol n-ario; cada hijo se construye llamando a la misma función
<code>con_totales</code>	Cada nodo suma lo de sus hijos	De abajo hacia arriba; devuelve un árbol nuevo

Recursividad + pureza: Fibonacci memoizado

Fibonacci recursivo "ingenuo" recalcula los mismos valores muchísimas veces: para `fib(20)` hace **21,891 llamadas**. Con `lru_cache` (memoización) guarda cada resultado y solo hace **21**. Esto **solo es correcto porque la función es pura**: si el resultado dependiera de algo externo, el valor guardado podría estar mal.

RESPUESTA MODELO · "¿POR QUÉ USASTE RECURSIVIDAD EN LOS ÁRBOLES?"

"Porque un árbol es una estructura recursiva: cada subárbol es otro árbol. Entonces la solución se escribe igual que la definición: el caso base es el árbol vacío, y el caso recursivo procesa el nodo y llama a la misma función con la izquierda y la derecha. Además, como los nodos son tuplas, insertar devuelve un árbol nuevo sin modificar el original."

EJERCICIO 5

- a) En `altura(arbol)`, ¿cuál es el caso base y cuál el recursivo?
- b) ¿Qué pasaría si borras las líneas `if n == 0: return 1` de `factorial`?
- c) ¿Por qué `inorden` entrega los precios ordenados?

Búscalo en el código: [\[PF recursividad\]](#) · [\[PF 2.6 árbol\]](#)

Lección 6 · Evaluación perezosa y generadores

Evaluación **ansiosa**: calcula todo de inmediato y lo guarda. Evaluación **perezosa**: calcula cada valor **solo cuando alguien lo pide**.

Python no es perezoso como Haskell, pero tiene herramientas perezosas: `range`, `map`, `filter`, los generadores con `yield` y el módulo `itertools`.

range: un intervalo que no guarda sus elementos (tema 2.3)

```
sys.getsizeof(range(1, 1_000_000_000))    # 48 bytes
sys.getsizeof(list(range(1_000_000)))      # ≈ 8 MB
```

El `range` solo guarda inicio, fin y paso. En la app se usa para generar los meses (`meses_entre`) y para paginar tablas (`indices_pagina`).

yield: una función que se pausa (diapositivas 26 y 27)

```
def numeros():
    n = 1
    while True:
        yield n          # entrega n y se PAUSA aquí
        n += 1          # el siguiente next() continúa desde esta línea

g = numeros()
next(g)    # 1
next(g)    # 2
```

El `while True` no congela el programa porque la función **se detiene en cada `yield`** hasta que alguien pide otro valor. La secuencia es infinita pero **nunca se guarda completa**.

Dónde se usa en VentasFx (core/perezoso.py)

Función	Qué hace	Por qué es perezosa
<code>folios()</code>	F-0000001, F-0000002, ... sin fin	<code>map</code> sobre <code>count()</code> : cada folio se crea al pedirlo
<code>simular_ventas()</code>	Flujo de ventas simuladas	Con días = infinito (<code>count</code>); con número de días, finito (<code>range</code>)
<code>leer_filas()</code> / <code>leer_ventas()</code>	Lee el CSV	Línea por línea con <code>yield from</code> y <code>map</code> : nunca carga el archivo completo
<code>tomar(n, flujo)</code>	Los primeros n elementos	<code>islice</code> deja de pedir al llegar a n, aunque el flujo sea infinito
<code>hasta_alcanzar()</code>	Ventas hasta llegar a una meta	<code>takewhile</code> corta el flujo infinito en el momento justo
<code>en_lotes()</code>	Agrupar en lotes de 10,000	Cada lote se arma cuando se pide

La demostración de la pantalla Carga masiva

```
def total_ansioso(ruta):
    todas = list(leer_ventas(ruta))    # carga TODO en memoria
    return total_general(todas)

def total_perezoso(ruta):
    return total_general(leer_ventas(ruta))    # una venta a la vez
```

Mismo total, pero con 250,000 ventas la versión ansiosa ocupó cerca de 148 MB y la perezosa alrededor de 0.1 MB. En "Leer solo lo necesario", `filter` + `islice` encuentran 10 ventas y dejan de leer: el resto del archivo nunca se toca.

RESPUESTA MODELO · "¿CÓMO EXISTE UNA SECUENCIA INFINITA SI LA MEMORIA ES FINITA?"

"Porque no se guarda completa. Un generador produce cada valor solo cuando se le pide con `next()` y se pausa en el `yield`. En mi app, `simular_ventas()` sin número de días es infinita, pero solo existe la venta que se está pidiendo. Funciones como `islice` o `takewhile` deciden cuándo dejar de pedir."

EJERCICIO 6

- a) ¿Qué devuelve `tomar(3, folios())`?
- b) ¿Qué pasaría si escribieras `list(numeros())`?
- c) ¿Por qué `leer_ventas` devuelve un `map` y no una lista?

Búscalo en el código: [PF 2.7 generador] · [PF 2.7 perezoso] · [PF 2.3 range]

Lección 7 · El estado de la app y las preguntas difíciles

Núcleo funcional, capa imperativa delgada. Toda la lógica está en `core/` (puro). Los efectos —pantalla, archivos, clics— viven en la orilla.

El único estado mutable: `ui/estado.py`

Streamlit vuelve a ejecutar la página en cada clic, así que necesita recordar cosas entre ejecuciones (`st.session_state`). Ese es el único lugar con estado mutable, y aun así se usa de forma funcional:

```
class EstadoTienda(NamedTuple):      # una "foto" inmutable de la tienda
    catalogo: tuple
    inventario: dict
    ventas_nuevas: tuple
    descripcion: str

# Registrar una venta = crear un estado NUEVO (paginas/ventas.py)
registrar_estado(estado._replace(
    inventario=actualizar_stock(estado.inventario, venta.sku, -venta.cantidad),
    ventas_nuevas=estado.ventas_nuevas + (venta,),
))

def deshacer():
    st.session_state.historial.pop()  # el estado anterior sigue completo
```

El historial es una **pila de estados inmutables**. Ningún estado se modifica: cada acción agrega uno nuevo. Por eso **deshacer es trivial**.

Los efectos que existen, y por qué están bien

Dónde	Efecto	Por qué es aceptable
<code>ui/estado.py</code>	<code>session_state</code>	Streamlit lo exige; se usa como pila de valores inmutables
<code>leer_filas</code> , <code>escribir_ventas</code>	Leer y escribir archivos	E/S aislada en funciones de la orilla
<code>cronometrar</code> , <code>medir_memoria</code>	Reloj y memoria	Solo miden, no calculan reglas de negocio
<code>reportes._sumar_en</code>	Modifica un dict local	Mutación local controlada; <code>agrupar</code> sigue pura hacia afuera
<code>simular_ventas</code>	Números aleatorios	Semilla fija: mismos datos siempre (reproducible)

Todos están marcados con **[PF efecto]**. Conocerlos de antemano te permite explicarlos antes de que te los señalen.

Preguntas difíciles y cómo responderlas

Pregunta	Respuesta corta
¿Tu programa es 100% funcional?	No, y no puede serlo: un programa útil necesita efectos (mostrar cosas, leer archivos). Lo que hice fue concentrar la lógica en funciones puras y aislar los efectos en la orilla. Python es multiparadigma; la presentación lo aclara.
¿Por qué el funcional a veces es más lento en el Laboratorio?	Algunas versiones recorren los datos más de una vez o crean objetos nuevos. A cambio se gana código fácil de probar y componer. Lo muestro sin esconderlo.
¿Por qué no usaste <code>for</code> ?	Sí hay <code>for</code> dentro de comprensiones y generadores. La idea no es prohibirlo, sino expresar el problema como transformaciones (diapositiva 5: "no se trata de prohibir <code>for</code> o variables").
¿Qué pasa si quito <code>frozen=True</code> ?	El programa seguiría funcionando, pero ya nada impediría modificar un producto por error. La protección pasaría a depender de la disciplina del programador.
¿Cómo pruebas que es puro?	Con <code>pytest</code> : 47 pruebas. Una llama 100 veces a <code>total_venta</code> con la misma venta y verifica un solo resultado; otras comprueban que el original no cambia.
¿Por qué <code>map/filter</code> y no solo comprensiones?	Uso ambas. El Laboratorio genera el mismo flujo en los dos estilos. La comprensión es más idiomática; <code>map/filter</code> hacen visible el concepto.
¿De dónde salen los datos?	Son simulados con un generador y semilla fija, con temporadas: Electrónica sube en diciembre y Papelería en agosto.

EJERCICIO 7

El docente señala `st.session_state.historial.append(nuevo)` y dice: "Aquí estás mutando una lista. ¿No que era funcional?" ¿Qué respondes?

Soluciones de los ejercicios

Ejercicio 4 · Suma de totales mayores a \$1,000

```
totales = [500, 1500, 3000, 800]          # (map: venta → total, ya hecho)
mayores = filter(lambda t: t > 1000, totales)  # filter → 1500, 3000
reduce(lambda a, b: a + b, mayores, 0)      # reduce → 4500
```

Primero **map** convierte cada venta en su total; **filter** se queda con 1,500 y 3,000; **reduce** los suma: **\$4,500**. En comprensión: `sum(t for t in totales if t > 1000)`.

Ejercicio 5 · Recursividad

- **a)** Caso base: `if arbol is None: return 0` (árbol vacío, altura 0). Caso recursivo: `1 + max(altura(izquierda), altura(derecha))`.
- **b)** Nunca se detendría: `factorial(0)` llamaría a `factorial(-1)` y así sin fin, hasta que Python lance `RecursionError`.
- **c)** En un árbol de búsqueda, todo lo de la izquierda es menor y todo lo de la derecha es mayor. Inorden visita izquierda → raíz → derecha, así que sale de menor a mayor: `[2, 5, 7, 10, 12, 15, 20]`.

Ejercicio 6 · Perezoso

- **a)** `('F-00000001', 'F-00000002', 'F-00000003')`. `islice` pide solo 3 aunque `folios()` sea infinito.
- **b)** Nunca terminaría: `list` intenta guardar todos los elementos de una secuencia infinita y la memoria se agotaría.
- **c)** Para no cargar el archivo completo: cada fila se convierte en `Venta` solo cuando alguien la pide. Así se pueden totalizar cientos de miles de ventas con muy poca memoria.

Ejercicio 7 · El `append` en `session_state`

RESPUESTA MODELO

"Sí, es la única mutación de la interfaz y es intencional: Streamlit necesita guardar algo entre clics. Pero lo que guardo son estados inmutables. Nunca modifico un estado: cada acción crea un `EstadoTienda` nuevo con `_replace` y lo agrego a la pila. Por eso deshacer es simplemente quitar el último. Toda la lógica de negocio está en `core/` y es pura; el efecto queda aislado en la capa de interfaz."

Hoja de repaso rápido

Concepto	En una frase	Ejemplo en VentasFx
Inmutabilidad	No se modifica: se crea un valor nuevo	<code>Producto(frozen=True)</code> , <code>Venta(NamedTuple)</code> , <code>ajustar_precio</code>
Función pura	Misma entrada → misma salida, sin efectos	<code>total_venta</code> , <code>utilidad</code> , <code>tasa_comision</code>
2.1 Tipos	Cada estructura elegida por lo que debe permitir	tuple (fijo), dict (búsqueda), set (únicos)
2.2 Primera clase	Las funciones son valores	<code>OPERADORES</code> , <code>PREDICADOS</code> , <code>PROBLEMAS</code>
2.2 Orden superior	Recibe o devuelve funciones	<code>regla()</code> , <code>y_()</code> , <code>compose()</code> , <code>clave_categoria()</code>
2.2 lambda	Función anónima de una expresión	<code>lambda venta:</code> <code>comparar(extraer(venta),</code> <code>valor)</code>
2.3 range	Intervalo que no guarda elementos	<code>meses_entre</code> , <code>indices_pagina</code>
2.4 operator / predicado	Operadores como funciones; funciones Sí/No	<code>operator.gt</code> , <code>itemgetter</code> , <code>es_par</code>
2.5 map	Transforma cada elemento	<code>map(total_venta, ventas)</code>
2.5 filter	Selecciona con un predicado	<code>filter(predicado, ventas)</code>
2.5 reduce	Combina muchos en uno	<code>resumen (5 KPIs)</code> , <code>total_general</code>
Recursividad	Caso base + caso recursivo más pequeño	<code>factorial</code> , <code>suma_arbol</code> , <code>con_totales</code>
2.6 Árboles	Nodo = (valor, izquierda, derecha)	BST por precio, árbol de categorías
2.7 Perezoso / yield	Se calcula solo lo que se pide	<code>numeros()</code> , <code>folios()</code> , <code>leer_ventas</code> , <code>islice</code>
Efecto aislado	Lo impuro vive en la orilla	<code>session_state</code> , archivos, reloj

Las 3 frases que debes poder decir sin pensar

1. "Mi sistema tiene un **núcleo funcional** en `core/` y una **capa imperativa delgada** en la interfaz."
2. "Todo sigue el flujo **datos → filter → map → reduce → resultado**."
3. "Los datos son **inmutables**: cada acción crea un valor nuevo, por eso puedo deshacer y compartir datos sin riesgo."

Atajos en github.dev

- `Ctrl + Shift + F` → buscar `[PF` (todas las marcas) o una marca concreta, por ejemplo `[PF 2.5 reduce]`.
- `Ctrl + P` → abrir un archivo por nombre (`arbol.py` , `perezoso.py` ...).
- Ten la app en línea en otra pestaña para mostrar el resultado de cada código.